

AI Engineering - Industry Roadmap

Continuation after Projects 01-07 | Focus: advanced RAG, agent systems, frameworks, evaluation, observability, and deployment

Goal: Move from understanding core AI application mechanics to building systems that resemble real company AI platforms. Continue learning every layer; use libraries only after understanding what they abstract.

Project 08 - Advanced RAG System

- Improve the AI Knowledge Workspace instead of starting from zero.
- **Build:** page-aware chunks, document filters, similarity scores, hybrid search (vector + keyword), reranking, citations, and no-answer thresholds.
- **Learn:** retrieval quality, metadata, recall vs precision, rerankers, hallucination control, context-window management.
- **Libraries:** pgvector, OpenAI embeddings, optional Cohere/Jina reranker, optional LlamaIndex retrievers.
- **Deliverable:** reliable document Q&A; with measurable source quality.

Project 09 - RAG Framework Comparison

- Rebuild one small RAG feature using a framework and compare it with your manual implementation.
- **Build:** the same ingestion, retrieval, and answer pipeline using LangChain or LlamaIndex.
- **Learn:** loaders, splitters, vector stores, retrievers, chains, framework abstractions, vendor lock-in.
- **Deliverable:** short comparison: manual FastAPI RAG vs framework-based RAG; keep the version you understand and can maintain.

Project 10 - Tool-Calling Research Agent

- Build one agent that chooses tools instead of following one fixed RAG pipeline.
- **Tools:** search_documents, list_documents, summarize_document, compare_documents, fetch_conversation_history.
- **Learn:** tool schemas, structured arguments, agent loop, tool results, limits, retries, permissions, prompt injection risks.
- **Libraries:** first implement a small manual tool loop; then repeat with the OpenAI Agents SDK.
- **Deliverable:** an agent that can answer, summarize, and compare documents using the correct tool.

Project 11 - Stateful Agent Workflows with LangGraph

- Turn an open-ended agent into an explicit, controlled workflow.
- **Workflow:** classify request -> retrieve -> evaluate evidence -> answer -> verify -> retry or finish.
- **Learn:** graph state, nodes, edges, conditional routing, checkpoints, memory, human approval, resumable execution.
- **Library:** LangGraph; use LangChain model/tool components only where useful.
- **Deliverable:** a persistent agent workflow that can pause, resume, and show every decision.

Project 12 - AI Evaluation and Observability

- Stop judging quality only by manually trying prompts.
- **Build:** evaluation dataset, retrieval hit-rate checks, answer relevance, faithfulness, tool-call correctness, latency and cost tracking.
- **Learn:** offline evals, regression tests, LLM-as-judge, deterministic checks, traces, spans, token usage, failure analysis.
- **Libraries:** Arize Phoenix or LangSmith; optional Ragas/DeepEval for evaluation experiments.
- **Deliverable:** dashboard plus a repeatable evaluation command that detects regressions.

Project 13 - Production Agent API

- Expose the agent through a company-style backend.
- **Build:** streaming responses, background ingestion jobs, cancellation, idempotency, rate limits, retry policies, audit logs, API versioning.
- **Learn:** queues, workers, async boundaries, failure recovery, secrets, tenant isolation, cost controls.
- **Libraries:** FastAPI, Celery or Dramatiq, Redis, SQLAlchemy, Alembic, OpenTelemetry.
- **Deliverable:** robust API that handles slow AI work without blocking requests.

Project 14 - Model and Provider Abstraction

- Avoid coupling the application to one model provider.
- **Build:** one internal interface for chat, embeddings, structured outputs, and tool calling.
- **Learn:** model routing, fallbacks, timeouts, cost/latency trade-offs, local models, prompt compatibility.
- **Libraries:** direct provider SDKs first; optionally LiteLLM for a unified provider layer.
- **Deliverable:** switch between at least two models without changing business logic.

Project 15 - Enterprise Integrations and MCP

- Connect the agent to real systems while preserving permissions.
- **Build:** connectors for email, calendar, files, GitHub, Slack, or internal APIs; expose selected capabilities as tools.
- **Learn:** OAuth, scopes, least privilege, Model Context Protocol (MCP), tool discovery, confirmation before writes, auditability.
- **Deliverable:** one secure read workflow and one confirmed write workflow.

Project 16 - Capstone: AI Operations Workspace

- Combine the strongest parts of the roadmap into one portfolio-grade system.
- **Capabilities:** authenticated workspaces, document RAG, agent tools, LangGraph workflow, streaming, background jobs, traces, evals, citations, deployment.
- **Architecture:** React + FastAPI, PostgreSQL/pgvector, Redis, object storage (S3/MinIO), worker service, observability platform.
- **Deployment:** Docker Compose locally; then cloud deployment with CI/CD and environment-based configuration.
- **Deliverable:** documented architecture, demo video, evaluation report, cost report, and production runbook.

Framework rule: Do not use LangChain, LangGraph, LlamaIndex, or an Agents SDK merely to shorten code. Use a framework when it gives a concrete benefit: reusable integrations, state, tracing, checkpointing, evaluation, or safer orchestration.

Stage	Primary focus	Recommended tool
08	Advanced retrieval	pgvector + direct SDK
09	Framework abstraction	LangChain or LlamaIndex
10	Agent fundamentals	Manual loop -> OpenAI Agents SDK
11	Stateful orchestration	LangGraph
12	Quality and debugging	Phoenix or LangSmith
13-16	Production platform	FastAPI, workers, Redis, S3, OpenTelemetry

Learning rule: Keep the original 20/80 balance: approximately 20% documentation and 80% building, debugging, tracing, evaluating, and improving. Finish one project before adding another framework.

Reference direction: official LangChain/LangGraph documentation, OpenAI Agents SDK documentation, and Arize Phoenix documentation were used to align the roadmap with current agent orchestration, tracing, and evaluation capabilities (August 2026).